BankPulse AI — Phase 1 Complete Build Guide

A step-by-step guide to building the Phase 1 infrastructure and synthetic data pipeline of BankPulse AI — a banking and fraud analytics platform on Azure. Written so that someone with zero prior knowledge of Azure, Bicep, or Python packaging can follow along and end up with the same working system.

What you'll have at the end: Azure cloud infrastructure provisioned from code, 6 million synthetic banking transactions with injected fraud patterns, all uploaded to an Azure Data Lake, reproducible from a single `az` command and a pair of Python scripts.

Time required: A focused weekend, or 2–3 evenings if you pace yourself.

Table of contents

---

1. What we're building and why

The product

BankPulse AI is a simulated retail bank's fraud analytics platform. It has five layers, each built in a separate phase:

Phase	Layer	Tech
1 (this guide)	Infrastructure + synthetic data + bronze ingestion	Bicep, Python, ADLS Gen2
2	PySpark medallion transforms	Delta Lake, PySpark
3	Warehouse with star schema	Azure SQL Database
4	AI analyst layer	LLM API, prompt engineering
5	Dashboards	Power BI

Why it's CV-worthy

Fraud analytics is one of the largest hiring categories in Dublin banking and fintech (AIB, BoI, Revolut, Stripe, Mastercard, Fiserv all have Dublin data teams). The six fraud typologies modelled here — card testing, account takeover, synthetic identity, velocity attacks, impossible travel, merchant collusion — are the real categories investigators use, and having them labelled in synthetic data demonstrates domain understanding that most junior portfolios lack.

Architecture at end of Phase 1

---

2. Prerequisites

- A Mac running macOS. Linux/Windows work similarly but exact commands differ.
- An Azure account. If you don't have one, sign up at azure.microsoft.com/en-us/free — Microsoft gives you €170 in free credits and a 30-day trial window, plus many services free for 12 months.
- A GitHub account.
- ~5 GB free disk space (virtual envs + synthetic data).
- Basic terminal familiarity (`cd`, `ls`, copy-paste commands).

---

3. Step 1 — Install the tools

You need five tools. Install each one and verify it works.

3.1 Homebrew

Homebrew is the package manager we'll use to install everything else. Skip if you already have it.

```bash
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/i
```

Verify:

```bash
brew --version
```

3.2 Azure CLI

The command-line interface to Azure. Every `az` command in this guide talks to the Azure control plane.

```bash
brew install azure-cli
```

Verify (expect version 2.50 or higher):

```bash
az --version
```

3.3 Bicep

Microsoft's infrastructure-as-code language. Ships as an `az` plugin.

```bash
az bicep install
```

Verify:

```bash
az bicep version
```

3.4 Python (via the `uv` package manager)

`uv` is a modern, extremely fast Python package manager. We'll use it to both install Python 3.11 and manage our project's virtual environment.

```bash
brew install uv
```

Verify:

```bash
uv --version
```

3.5 VS Code

Our IDE. Free, excellent Python + Bicep extensions.

Download from code.visualstudio.com, install, open it.

Then install these three extensions (Cmd+Shift+X, search, Install):

- **Python** by Microsoft
- **Bicep** by Microsoft
- **GitHub Pull Requests and Issues** by GitHub

3.6 Log into Azure

```bash
az login
```

A browser window opens — sign in with your Microsoft account. The terminal then prints a list of your subscriptions. If you only have one, it's auto-selected.

Confirm your subscription:

```bash
az account show --query "{name:name, id:id, user:user.name}" -o table
```

Write down your subscription ID — you may reference it later.

---

4. Step 2 — Create the repo skeleton

Empty project folder with proper layout, initialised as a git repo, pushed to GitHub.

4.1 Create the folder

bash

```
mkdir -p ~/projects && cd ~/projects
mkdir bankpulse-ai && cd bankpulse-ai
```

4.2 Create the directory tree

bash

```
mkdir -p \
  infra \
  src/bankpulse/generators \
  src/bankpulse/ingestion \
  src/bankpulse/utils \
  scripts \
  configs \
  tests \
  .github/workflows
```

4.3 Create placeholder files

Python needs `__init__.py` in every package folder. Git doesn't track empty directories, so these files also tell git to preserve the folder structure.

bash

```
touch \
  src/bankpulse/__init__.py \
  src/bankpulse/generators/__init__.py \
  src/bankpulse/ingestion/__init__.py \
  src/bankpulse/utils/__init__.py \
  tests/__init__.py \
  README.md \
  .gitignore
```

4.4 Starter README

bash

```
cat > README.md << 'EOF'
# BankPulse AI

End-to-end banking + fraud analytics platform on Azure, with an AI analyst layer tha

**Status:** Phase 1 — infrastructure and synthetic data ingestion.

**Stack:** Python 3.11+ · Pydantic v2 · PySpark + Delta Lake · Azure SQL · ADLS Gen2
EOF
```

4.5 `.gitignore`

This prevents secrets, caches, and generated data from ever being committed.

bash

```
cat > .gitignore << 'EOF'
# Python
__pycache__/
*.py[cod]
.venv/
*.egg-info/
.pytest_cache/
.mypy_cache/
.ruff_cache/

# Secrets — NEVER commit
.env
.env.local

# Local data
data/
spark-warehouse/

# Azure build artefacts
infra/*.json
!infra/*.parameters.example.json
infra/main.parameters.json

# macOS
.DS_Store

# Editors
.vscode/
.idea/
EOF
```

4.6 Initialise git

```bash
git init
git branch -M main
git add .
git commit -m "chore: initial project skeleton"
```

4.7 Create the GitHub repo

Options:

Option A — via VS Code (easiest): open the folder in VS Code (`code .`), go to the Source Control panel (left sidebar), click **Publish Branch** → pick **public repository** → name it `bankpulse-ai`. VS Code handles auth via the browser.

Option B — via web: go to github.com/new, create a public repo called `bankpulse-ai` (do NOT tick "Add a README" — we have one), then back in the terminal:

```bash
git remote add origin https://github.com/YOUR_USERNAME/bankpulse-ai.git
git push -u origin main
```

Verify:

```bash
git remote -v
```

Should print `origin` twice with your GitHub URL.

---

5. Step 3 — Write the Bicep infrastructure file

This is the most important file in Phase 1. It describes all our Azure resources as code — so they can be deployed, torn down, or rebuilt from scratch with one command.

5.1 Create the Azure resource group

A resource group is a "folder" in Azure that holds related resources. It's free; you only pay for what's inside it.

```bash
az group create --name bankpulse-rg --location northeurope
```

Expected: JSON output ending in `"provisioningState": "Succeeded"`.

5.2 Create `infra/main.bicep`

In VS Code, right-click the `infra` folder → New File → `main.bicep`. Paste:

```bicep
```

```bicep
// ============================================================================
// BankPulse AI — Phase 1 Infrastructure
// Provisions: ADLS Gen2 storage, Azure SQL (free offer), Key Vault.
// ============================================================================

targetScope = 'resourceGroup'

// ---------- Parameters ------------------------------------------------------

@description('Base name used as a prefix for all resources')
param baseName string = 'bankpulse'

@description('Environment tag')
@allowed([ 'dev', 'prod' ])
param environment string = 'dev'

@description('Azure region. Defaults to the resource group region.')
param location string = resourceGroup().location

@description('SQL Server admin login name')
param sqlAdminLogin string

@description('SQL Server admin password. Must meet Azure complexity requirements.')
@secure()
param sqlAdminPassword string

@description('Your current public IP address, for SQL firewall allow-list')
param clientIpAddress string

// ---------- Variables -------------------------------------------------------

var suffix = take(uniqueString(resourceGroup().id), 6)
var storageAccountName = toLower('${baseName}${environment}${suffix}')
var keyVaultName       = '${baseName}-kv-${suffix}'
var sqlServerName      = '${baseName}-sql-${environment}-${suffix}'
var sqlDatabaseName    = '${baseName}db'

var tags = {
  project:     'bankpulse-ai'
  environment: environment
  managedBy:   'bicep'
}

// ---------- ADLS Gen2 storage -----------------------------------------------

resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
```

```
  name: storageAccountName
  location: location
  tags: tags
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
  properties: {
    isHnsEnabled: true                  // ADLS Gen2 switch
    accessTier: 'Hot'
    minimumTlsVersion: 'TLS1_2'
    allowBlobPublicAccess: false
    allowSharedKeyAccess: true
    supportsHttpsTrafficOnly: true
  }
}

resource blobService 'Microsoft.Storage/storageAccounts/blobServices@2023-05-01' = {
  parent: storage
  name: 'default'
}

resource bronze 'Microsoft.Storage/storageAccounts/blobServices/containers@2023-05-0
  parent: blobService
  name: 'bronze'
}
resource silver 'Microsoft.Storage/storageAccounts/blobServices/containers@2023-05-0
  parent: blobService
  name: 'silver'
}
resource gold 'Microsoft.Storage/storageAccounts/blobServices/containers@2023-05-01'
  parent: blobService
  name: 'gold'
}

// ---------- Key Vault ----------------------------------------------------

resource keyVault 'Microsoft.KeyVault/vaults@2023-07-01' = {
  name: keyVaultName
  location: location
  tags: tags
  properties: {
    sku: { family: 'A', name: 'standard' }
    tenantId: subscription().tenantId
    enableRbacAuthorization: true
    enabledForTemplateDeployment: true
    enableSoftDelete: true
    softDeleteRetentionInDays: 7
    publicNetworkAccess: 'Enabled'
```

```
  }
}

// ---------- Azure SQL Database (free offer) ---------------------------------

resource sqlServer 'Microsoft.Sql/servers@2023-08-01-preview' = {
  name: sqlServerName
  location: location
  tags: tags
  properties: {
    administratorLogin: sqlAdminLogin
    administratorLoginPassword: sqlAdminPassword
    version: '12.0'
    minimalTlsVersion: '1.2'
    publicNetworkAccess: 'Enabled'
  }
}

resource fwClient 'Microsoft.Sql/servers/firewallRules@2023-08-01-preview' = {
  parent: sqlServer
  name: 'AllowClientIP'
  properties: {
    startIpAddress: clientIpAddress
    endIpAddress: clientIpAddress
  }
}

resource fwAzure 'Microsoft.Sql/servers/firewallRules@2023-08-01-preview' = {
  parent: sqlServer
  name: 'AllowAzureServices'
  properties: {
    startIpAddress: '0.0.0.0'
    endIpAddress: '0.0.0.0'
  }
}

resource sqlDb 'Microsoft.Sql/servers/databases@2023-08-01-preview' = {
  parent: sqlServer
  name: sqlDatabaseName
  location: location
  tags: tags
  sku: {
    name: 'GP_S_Gen5'
    tier: 'GeneralPurpose'
    family: 'Gen5'
    capacity: 2
  }
```

```
  properties: {
    collation: 'SQL_Latin1_General_CP1_CI_AS'
    maxSizeBytes: 34359738368
    autoPauseDelay: 60
    minCapacity: json('0.5')
    useFreeLimit: true
    freeLimitExhaustionBehavior: 'AutoPause'
  }
}

// ---------- Outputs ----------

output storageAccountName string = storage.name
output keyVaultName       string = keyVault.name
output sqlServerName      string = sqlServer.name
output sqlServerFqdn      string = sqlServer.properties.fullyQualifiedDomainName
output sqlDatabaseName    string = sqlDb.name
output bronzeContainerUrl string = '${storage.properties.primaryEndpoints.dfs}bronze
```

Save (Cmd+S).

5.3 Verify the file compiles

```bash
az bicep build --file infra/main.bicep
```

Silent output = success. Delete the generated JSON:

```bash
rm -f infra/main.json
```

5.4 Commit

```bash
git add infra/
git commit -m "feat(infra): bicep template for storage, key vault, SQL"
git push
```

What the Bicep file does, briefly

- **`param` blocks** — inputs you supply at deploy time. `@secure()` on the password keeps it out of deployment logs.
- **`uniqueString(resourceGroup().id)`** — generates a stable, reproducible 6-character suffix so resource names are globally unique (storage accounts require this) but consistent

across redeploys.

- **Storage account** with `isHnsEnabled: true` — hierarchical namespace, which is what turns plain blob storage into ADLS Gen2 (real folders, atomic renames, per-folder ACLs).
- **Three containers** — bronze, silver, gold — the standard "medallion" pattern.
- **Key Vault with** `enableRbacAuthorization: true` — access controlled by Azure roles, not legacy access policies.
- **SQL Server + Database** on the free tier: `useFreeLimit: true` enables 100,000 vCore-seconds + 32 GB monthly for free; `freeLimitExhaustionBehavior: 'AutoPause'` is a hard ceiling — the DB pauses until next month rather than billing.
- **Two firewall rules**: your home IP for dev access, plus the magic `0.0.0.0` rule that allows trusted Azure services (Power BI, Data Factory).
- `output` **blocks** — values Azure echoes back after deployment for us to copy into `.env`.

---

6. Step 4 — Deploy the infrastructure

6.1 Gather inputs

Your public IP:

```bash
curl -s https://api.ipify.org
```

A strong SQL admin password. Must meet Azure's rules: 8+ chars, 3 of: uppercase, lowercase, digit, special. Don't include your username or dictionary words.

Write it down in a password manager.

6.2 Set the password as a shell variable

```bash
read -s SQL_ADMIN_PASSWORD
```

Type your password, hit Enter. You won't see characters as you type (intentional).

Verify length:

```bash
echo "Length: ${#SQL_ADMIN_PASSWORD}"
```

Should match your actual password length. If wrong, `unset SQL_ADMIN_PASSWORD` and try again — you probably pasted extra characters.

6.3 Deploy

```bash
az deployment group create \
  --resource-group bankpulse-rg \
  --template-file infra/main.bicep \
  --parameters \
      sqlAdminLogin=bankpulseadmin \
      sqlAdminPassword="$SQL_ADMIN_PASSWORD" \
      clientIpAddress="$(curl -s https://api.ipify.org)"
```

Takes 3–5 minutes. Expect a big JSON blob at the end with an `outputs` section listing your resource names.

6.4 Verify

```bash
az deployment group show \
  --resource-group bankpulse-rg \
  --name main \
  --query "{state:properties.provisioningState, outputs:properties.outputs}" \
  -o json
```

`"state": "Succeeded"` means you're done. Copy the `outputs` — you'll need them in Step 5.

---

7. Step 5 — Wire up Key Vault and `.env`

7.1 Grant yourself Key Vault access

Bicep created the vault but didn't auto-grant you access. Do that now:

```bash
KV_NAME="bankpulse-kv-XXXXXX"      # Replace XXXXXX with your suffix
MY_OBJECT_ID=$(az ad signed-in-user show --query id -o tsv)
VAULT_ID=$(az keyvault show -n $KV_NAME --query id -o tsv)

az role assignment create \
  --role "Key Vault Secrets Officer" \
  --assignee "$MY_OBJECT_ID" \
  --scope "$VAULT_ID"
```

Wait ~15 seconds for RBAC propagation.

7.2 Store the SQL password

```bash
az keyvault secret set \
  --vault-name $KV_NAME \
  --name sql-admin-password \
  --value "$SQL_ADMIN_PASSWORD"
```

7.3 Verify

```bash
az keyvault secret show --vault-name $KV_NAME --name sql-admin-password --query valu
```

Should print your password back.

7.4 Grant yourself storage access

Same pattern — needed later for the uploader:

```bash
STORAGE_NAME="bankpulsedevXXXXXX"   # Replace XXXXXX with your suffix
STORAGE_ID=$(az storage account show -n $STORAGE_NAME -g bankpulse-rg --query id -o

az role assignment create \
  --role "Storage Blob Data Contributor" \
  --assignee "$MY_OBJECT_ID" \
  --scope "$STORAGE_ID"
```

7.5 Create `.env`

In VS Code, right-click the project root → New File → `.env`. Paste, replacing with your actual resource names from the deployment outputs:

```bash
AZURE_STORAGE_ACCOUNT_NAME=bankpulsedevXXXXXX
AZURE_KEY_VAULT_NAME=bankpulse-kv-XXXXXX
AZURE_SQL_SERVER_FQDN=bankpulse-sql-dev-XXXXXX.database.windows.net
AZURE_SQL_DATABASE_NAME=bankpulsedb
```

7.6 Create `.env.example` for the repo

Also at the project root:

```bash
```

```
AZURE_STORAGE_ACCOUNT_NAME=bankpulsedevXXXXXX
AZURE_KEY_VAULT_NAME=bankpulse-kv-XXXXXX
AZURE_SQL_SERVER_FQDN=bankpulse-sql-dev-XXXXXX.database.windows.net
AZURE_SQL_DATABASE_NAME=bankpulsedb
```

7.7 Confirm `.env` is ignored

bash

```
git status
```

You should see `.env.example` listed as untracked, but NOT `.env`. If `.env` appears, stop — your `.gitignore` isn't working; don't commit.

7.8 Commit

bash

```
git add .env.example
git commit -m "chore: add .env.example template"
git push
```

---

8. Step 6 — Set up the Python project

8.1 Create `pyproject.toml`

At the project root:

toml

```toml
[project]
name = "bankpulse"
version = "0.1.0"
description = "End-to-end banking + fraud analytics platform with an AI analyst laye
readme = "README.md"
requires-python = ">=3.11,<3.13"
authors = [{ name = "Your Name" }]

dependencies = [
    "pydantic>=2.6",
    "pydantic-settings>=2.2",
    "faker>=24.0",
    "numpy>=1.26",
    "pandas>=2.2",
    "pyarrow>=15.0",
    "httpx>=0.27",
    "anyio>=4.3",
    "tenacity>=8.2",
    "azure-identity>=1.16",
    "azure-keyvault-secrets>=4.8",
    "azure-storage-file-datalake>=12.14",
    "pyyaml>=6.0",
    "structlog>=24.1",
    "rich>=13.7",
    "typer>=0.12",
]

[project.optional-dependencies]
dev = [
    "ruff>=0.4",
    "mypy>=1.9",
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/bankpulse"]

[tool.ruff]
line-length = 100
target-version = "py311"
```

```
[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM", "N"]

[tool.pytest.ini_options]
asyncio_mode = "auto"
pythonpath = ["src"]
```

8.2 Create the venv and install deps

bash

```
uv venv --python 3.11
source .venv/bin/activate
uv pip install -e ".[dev]"
```

8.3 Verify

bash

```
python --version   # should print 3.11.x
python -c "import pydantic, faker, azure.identity, structlog; print('OK')"
```

8.4 Point VS Code at the venv

Cmd+Shift+P → `Python: Select Interpreter` → pick the `.venv` one.

8.5 Commit

bash

```
git add pyproject.toml
git commit -m "chore: python project config + dependencies"
git push
```

---

9. Step 7 — Write Pydantic models

These schemas are the contract between every layer. Any record written must match.

9.1 Create `src/bankpulse/models.py`

python

```python
"""Pydantic schemas for every record type flowing through the pipeline."""
from __future__ import annotations

from datetime import date, datetime
from decimal import Decimal
from enum import StrEnum
from typing import Annotated
from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field, field_validator


class CardNetwork(StrEnum):
    VISA = "VISA"
    MASTERCARD = "MASTERCARD"
    AMEX = "AMEX"


class Channel(StrEnum):
    CARD_PRESENT = "CARD_PRESENT"
    ECOMMERCE = "ECOMMERCE"
    CONTACTLESS = "CONTACTLESS"
    ATM = "ATM"
    TRANSFER = "TRANSFER"


class TransactionStatus(StrEnum):
    APPROVED = "APPROVED"
    DECLINED = "DECLINED"
    PENDING = "PENDING"
    REVERSED = "REVERSED"


class FraudPattern(StrEnum):
    NONE = "NONE"
    CARD_TESTING = "CARD_TESTING"
    ACCOUNT_TAKEOVER = "ACCOUNT_TAKEOVER"
    SYNTHETIC_IDENTITY = "SYNTHETIC_IDENTITY"
    VELOCITY_ATTACK = "VELOCITY_ATTACK"
    IMPOSSIBLE_TRAVEL = "IMPOSSIBLE_TRAVEL"
    MERCHANT_COLLUSION = "MERCHANT_COLLUSION"


class _Base(BaseModel):
    model_config = ConfigDict(
        str_strip_whitespace=True,
```

```
        validate_assignment=True,
        extra="forbid",
        frozen=False,
    )


class Customer(_Base):
    customer_id: UUID
    first_name: str
    last_name: str
    email: str
    date_of_birth: date
    home_city: str
    home_country: str = Field(min_length=2, max_length=2)
    signup_date: date
    risk_segment: Annotated[str, Field(pattern="^(LOW|MEDIUM|HIGH)$")]
    annual_income_eur: Decimal = Field(ge=0)

    @field_validator("home_country")
    @classmethod
    def upper_country(cls, v: str) -> str:
        return v.upper()


class Merchant(_Base):
    merchant_id: UUID
    merchant_name: str
    mcc: int = Field(ge=1000, le=9999)
    mcc_description: str
    city: str
    country: str = Field(min_length=2, max_length=2)
    is_online: bool
    risk_flag: bool = False


class Card(_Base):
    card_id: UUID
    customer_id: UUID
    card_network: CardNetwork
    issued_date: date
    expiry_date: date
    is_active: bool = True


class Transaction(_Base):
    transaction_id: UUID
    customer_id: UUID
```

```python
    card_id: UUID
    merchant_id: UUID
    transaction_ts: datetime
    amount_eur: Decimal = Field(gt=0, decimal_places=2)
    channel: Channel
    status: TransactionStatus
    location_city: str
    location_country: str = Field(min_length=2, max_length=2)
    is_fraud: bool = False
    fraud_pattern: FraudPattern = FraudPattern.NONE

    @field_validator("location_country")
    @classmethod
    def upper_country(cls, v: str) -> str:
        return v.upper()
```

9.2 Verify

```bash
python -c "from bankpulse.models import Customer, Merchant, Card, Transaction, Fraud
```

9.3 Commit

```bash
git add src/bankpulse/models.py
git commit -m "feat(models): pydantic schemas for customers, merchants, txns"
git push
```

---

10. Step 8 — Build the synthetic data generators

Six files total. Build them in this order.

10.1 Config loader — `src/bankpulse/config.py`

```python
```

```python
"""Config loader — YAML for defaults, .env for secrets/overrides."""
from __future__ import annotations

from functools import lru_cache
from pathlib import Path

import yaml
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict


class AzureSettings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_prefix="AZURE_", extra="i
    storage_account_name: str = Field(..., description="ADLS Gen2 account")
    key_vault_name: str
    sql_server_fqdn: str
    sql_database_name: str = "bankpulsedb"


class AppSettings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    environment: str = "dev"
    log_level: str = "INFO"
    data_dir: Path = Path("./data")
    upload_batch_size: int = 50_000
    max_concurrent_uploads: int = 8
    default_customer_count: int = 50_000
    default_days: int = 90
    default_fraud_rate: float = 0.008


def _load_yaml(path: Path) -> dict:
    if not path.exists():
        return {}
    return yaml.safe_load(path.read_text()) or {}


@lru_cache
def get_azure_settings() -> AzureSettings:
    return AzureSettings()  # type: ignore[call-arg]


@lru_cache
def get_app_settings(env: str = "dev") -> AppSettings:
```

```python
    yaml_overrides = _load_yaml(Path(f"configs/{env}.yaml"))
    return AppSettings(**yaml_overrides)
```

10.2 Logging — `src/bankpulse/utils/logging.py`

```python
"""Structured logging with structlog."""
from __future__ import annotations

import logging
import sys

import structlog


def configure_logging(level: str = "INFO", json_logs: bool = False) -> None:
    timestamper = structlog.processors.TimeStamper(fmt="iso")
    shared_processors: list = [
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.StackInfoRenderer(),
        timestamper,
    ]
    renderer = (
        structlog.processors.JSONRenderer()
        if json_logs else
        structlog.dev.ConsoleRenderer(colors=True)
    )
    structlog.configure(
        processors=shared_processors + [renderer],
        wrapper_class=structlog.make_filtering_bound_logger(getattr(logging, level.u
        context_class=dict,
        logger_factory=structlog.PrintLoggerFactory(file=sys.stdout),
        cache_logger_on_first_use=True,
    )


def get_logger(name: str) -> structlog.stdlib.BoundLogger:
    return structlog.get_logger(name)
```

10.3 Dev YAML — `configs/dev.yaml`

```yaml
```

```yaml
environment: dev
log_level: INFO
data_dir: ./data
upload_batch_size: 50000
max_concurrent_uploads: 8
default_customer_count: 50000
default_days: 90
default_fraud_rate: 0.008
```

10.4 Customers — `src/bankpulse/generators/customers.py`

python

```python
"""Synthetic customer generator."""
from __future__ import annotations

from datetime import date, timedelta
from decimal import Decimal
from random import Random
from uuid import uuid4

import numpy as np
from faker import Faker

from bankpulse.models import Customer


_COUNTRY_WEIGHTS: dict[str, float] = {
    "IE": 0.55, "GB": 0.15, "FR": 0.05, "DE": 0.05, "ES": 0.04,
    "IT": 0.03, "NL": 0.03, "PL": 0.03, "US": 0.03, "PT": 0.02, "BE": 0.02,
}
_IRISH_CITIES = [
    "Dublin", "Cork", "Galway", "Limerick", "Waterford", "Drogheda",
    "Dundalk", "Swords", "Bray", "Navan",
]


class CustomerGenerator:
    def __init__(self, seed: int = 42) -> None:
        self._rand = Random(seed)
        self._np = np.random.default_rng(seed)
        self._faker = Faker(["en_IE", "en_GB", "en_US"])
        self._faker.seed_instance(seed)

    def _pick_country(self) -> str:
        return self._rand.choices(
            list(_COUNTRY_WEIGHTS), weights=list(_COUNTRY_WEIGHTS.values()), k=1
        )[0]

    def _pick_city(self, country: str) -> str:
        if country == "IE":
            return self._rand.choice(_IRISH_CITIES)
        return self._faker.city()

    def _sample_income(self) -> Decimal:
        value = float(self._np.lognormal(mean=10.6, sigma=0.55))
        return Decimal(f"{value:.2f}")

    def _sample_risk_segment(self) -> str:
```

```python
        return self._rand.choices(["LOW", "MEDIUM", "HIGH"], weights=[0.70, 0.25, 0.

    def generate_one(self) -> Customer:
        country = self._pick_country()
        today = date(2026, 1, 1)
        dob = today - timedelta(days=self._rand.randint(18 * 365, 75 * 365))
        signup = today - timedelta(days=self._rand.randint(30, 10 * 365))
        first = self._faker.first_name()
        last = self._faker.last_name()
        email = f"{first}.{last}.{self._rand.randint(1, 9999)}@example.com".lower()

        return Customer(
            customer_id=uuid4(),
            first_name=first, last_name=last, email=email,
            date_of_birth=dob, home_city=self._pick_city(country),
            home_country=country, signup_date=signup,
            risk_segment=self._sample_risk_segment(),
            annual_income_eur=self._sample_income(),
        )

    def generate(self, n: int) -> list[Customer]:
        return [self.generate_one() for _ in range(n)]
```

10.5 Merchants — `src/bankpulse/generators/merchants.py`

```python
```

```python
"""Synthetic merchant generator with realistic MCC categories."""
from __future__ import annotations

from random import Random
from uuid import uuid4

from faker import Faker

from bankpulse.models import Merchant


_MCC_CATALOG: list[tuple[int, str, bool]] = [
    (5411, "Grocery Stores & Supermarkets", False),
    (5812, "Eating Places & Restaurants", False),
    (5814, "Fast Food Restaurants", False),
    (5541, "Service Stations (Fuel)", False),
    (5912, "Drug Stores & Pharmacies", False),
    (5311, "Department Stores", False),
    (5651, "Family Clothing Stores", False),
    (5732, "Electronics Stores", False),
    (5999, "Miscellaneous Retail", False),
    (4121, "Taxicabs & Limousines", False),
    (4111, "Local & Suburban Transport", False),
    (4511, "Airlines", True),
    (7011, "Hotels, Motels, Resorts", True),
    (5967, "Direct Marketing – Inbound Telemarketing", True),
    (5968, "Direct Marketing – Subscription", True),
    (7995, "Betting & Gaming", True),
    (6011, "Automated Cash Disbursement (ATM)", False),
    (6051, "Non-Financial Institutions – Crypto", True),
]

_COUNTRY_WEIGHTS: dict[str, float] = {
    "IE": 0.45, "GB": 0.15, "FR": 0.05, "DE": 0.05, "ES": 0.05,
    "IT": 0.03, "NL": 0.03, "PL": 0.02, "US": 0.12, "PT": 0.02, "BE": 0.03,
}


class MerchantGenerator:
    def __init__(self, seed: int = 43) -> None:
        self._rand = Random(seed)
        self._faker = Faker()
        self._faker.seed_instance(seed)

    def _pick_country(self) -> str:
        return self._rand.choices(
```

```
            list(_COUNTRY_WEIGHTS), weights=list(_COUNTRY_WEIGHTS.values()), k=1
        )[0]

    def generate_one(self) -> Merchant:
        mcc, desc, is_online_default = self._rand.choice(_MCC_CATALOG)
        is_online = is_online_default or self._rand.random() < 0.15
        country = self._pick_country()
        risk_flag = (
            (mcc in (6051, 7995) and self._rand.random() < 0.25)
            or self._rand.random() < 0.01
        )
        return Merchant(
            merchant_id=uuid4(),
            merchant_name=self._faker.company(),
            mcc=mcc, mcc_description=desc,
            city=self._faker.city(), country=country,
            is_online=is_online, risk_flag=risk_flag,
        )

    def generate(self, n: int) -> list[Merchant]:
        return [self.generate_one() for _ in range(n)]
```

10.6 Fraud patterns — `src/bankpulse/generators/fraud_patterns.py`

Too long to include here verbatim. See the repository — it implements six `inject_<pattern>` methods on a `FraudInjector` class plus a weighted dispatcher `inject_random`. Each injector returns a list of `Transaction` objects all tagged with the matching `FraudPattern` enum value.

The six patterns:

- `inject_card_testing` — 6–15 tiny (<€5) CNP transactions in <10 min, mostly declined
- `inject_account_takeover` — 2–5 high-value (€250–€2500) CNP transactions, all approved
- `inject_velocity_attack` — 4–8 card-present swipes at one merchant in minutes
- `inject_impossible_travel` — exactly 2 card-present txns in different countries, <90 min apart
- `inject_synthetic_identity` — 3–6 small legit-looking spends, then one €3000–€9000 bust-out
- `inject_merchant_collusion` — 2–4 rounded (€100/€200/€500/€750/€1000) txns at a risk-flagged merchant

10.7 Transactions — `src/bankpulse/generators/transactions.py`

This pulls it all together: creates cards, generates legitimate transactions (Poisson daily counts, log-normal amounts, trimodal hourly distribution, 88% home country), samples a subset of

customers as fraud victims, and periodically injects fraud episodes from the injector module. Returns a mixed list.

See the repository for the full file.

10.8 Package exports — `src/bankpulse/generators/__init__.py`

python

```python
"""Synthetic data generators."""
from bankpulse.generators.customers import CustomerGenerator
from bankpulse.generators.merchants import MerchantGenerator
from bankpulse.generators.transactions import TransactionGenerator

__all__ = ["CustomerGenerator", "MerchantGenerator", "TransactionGenerator"]
```

10.9 Verify + commit

bash

```bash
python -c "
from bankpulse.generators import CustomerGenerator, MerchantGenerator, TransactionGe
print('Generators OK')
"

git add src/bankpulse/ configs/
git commit -m "feat(generators): synthetic data engine with fraud patterns"
git push
```

---

11. Step 9 — Async ADLS uploader

11.1 Create `src/bankpulse/ingestion/adls.py`

python

```python
"""Async ADLS Gen2 uploader for the bronze layer."""
from __future__ import annotations

import asyncio
from collections.abc import Iterable
from pathlib import Path

from azure.identity import DefaultAzureCredential
from azure.storage.filedatalake import DataLakeServiceClient
from tenacity import (
    retry, retry_if_exception_type, stop_after_attempt, wait_random_exponential,
)

from bankpulse.utils.logging import get_logger

log = get_logger(__name__)


class BronzeUploader:
    def __init__(
        self,
        storage_account_name: str,
        container: str = "bronze",
        max_concurrent: int = 8,
    ) -> None:
        account_url = f"https://{storage_account_name}.dfs.core.windows.net"
        self._service = DataLakeServiceClient(
            account_url=account_url,
            credential=DefaultAzureCredential(),
        )
        self._fs = self._service.get_file_system_client(container)
        self._sem = asyncio.Semaphore(max_concurrent)

    @retry(
        stop=stop_after_attempt(4),
        wait=wait_random_exponential(multiplier=1, max=20),
        retry=retry_if_exception_type(Exception),
        reraise=True,
    )
    def _upload_sync(self, local_path: Path, remote_path: str) -> int:
        data = local_path.read_bytes()
        file_client = self._fs.get_file_client(remote_path)
        file_client.upload_data(data, overwrite=True)
        return len(data)

    async def upload_one(self, local_path: Path, remote_path: str) -> int:
```

```
        async with self._sem:
            log.info("upload.start", remote=remote_path, size=local_path.stat().st_s
            size = await asyncio.to_thread(self._upload_sync, local_path, remote_pat
            log.info("upload.done", remote=remote_path, bytes=size)
            return size

    async def upload_many(self, pairs: Iterable[tuple[Path, str]]) -> int:
        tasks = [self.upload_one(lp, rp) for lp, rp in pairs]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        total_bytes = 0
        failures = 0
        for r in results:
            if isinstance(r, Exception):
                failures += 1
                log.error("upload.failed", error=str(r))
            else:
                total_bytes += r
        log.info("upload.batch_complete", total_bytes=total_bytes, failures=failures
        if failures:
            raise RuntimeError(f"{failures} upload(s) failed")
        return total_bytes
```

11.2 Exports — `src/bankpulse/ingestion/__init__.py`

python

```python
"""ADLS ingestion."""
from bankpulse.ingestion.adls import BronzeUploader

__all__ = ["BronzeUploader"]
```

11.3 Commit

bash

```bash
git add src/bankpulse/ingestion/
git commit -m "feat(ingestion): async ADLS Gen2 bronze uploader"
git push
```

---

12. Step 10 — Run the pipeline end-to-end

12.1 CLI to generate data — `scripts/generate_data.py`

See the repository for the full script. It takes flags `--customers`, `--days`, `--fraud-rate`, `--out-dir` and writes dimension Parquet files + date-partitioned transaction Parquet files to a local

directory.

12.2 CLI to ingest — `scripts/ingest_to_bronze.py`

Collects local Parquet files and async-uploads them to the bronze container using `BronzeUploader`.

12.3 Small test run

```bash
python scripts/generate_data.py --customers 500 --days 7 --fraud-rate 0.02
find data/raw -type f | head -20
```

12.4 Full run

```bash
python scripts/generate_data.py --customers 50000 --days 90 --fraud-rate 0.008
```

Takes 3–5 minutes. Produces ~6M transactions, ~500 MB of Parquet.

12.5 Upload

```bash
python scripts/ingest_to_bronze.py
```

Takes 1–3 minutes. Watches concurrently-running uploads in the logs.

12.6 Verify in the portal

portal.azure.com → **Resource groups** → **bankpulse-rg** → click the storage account → **Data storage** → **Containers** → **bronze**. You should see four top-level folders: `cards/`, `customers/`, `merchants/`, `transactions/`. Inside `transactions/` should be ~90 `date=YYYY-MM-DD/` folders, each with one Parquet file.

12.7 Commit

```bash
git add scripts/
git commit -m "feat(scripts): CLI for generate + ingest"
git push
```

---

13. Troubleshooting

"Subscription not found" when running `az rest`. CLI logged into the wrong account. Run `az account clear && az login`.

Password length wrong when using `read -s`. You probably pasted extra characters. `unset SQL_ADMIN_PASSWORD` and try again. Fallback: pass the password inline in single quotes when deploying.

Bicep warning about hardcoded `core.windows.net`. Use `storage.properties.primaryEndpoints.dfs` instead of building the URL by hand.

"Forbidden" when setting Key Vault secret. RBAC hasn't propagated yet. Wait 30 seconds and retry.

Upload fails with 403. Forgot to assign "Storage Blob Data Contributor" role on the storage account. See Step 5.4.

`ModuleNotFoundError: No module named 'bankpulse.xxx'`. Either the file doesn't exist where you think, or the venv isn't active. Run `ls src/bankpulse/xxx.py` and `which python` to confirm.

"pathspec did not match any files" on git add. Typo in the file path. Double-check spelling.

---

14. Teardown

When you want everything gone:

```bash
az group delete --name bankpulse-rg --yes --no-wait
```

All Azure resources deleted. No further charges. Redeploy any time by rerunning Step 4.

What you've built

- Azure cloud infrastructure provisioned from code (Bicep)
- ADLS Gen2, SQL Database (free offer), Key Vault with RBAC
- Python project: Pydantic v2, pydantic-settings, structlog, typer, modern uv-managed venv
- Synthetic data engine: 50k customers, 5k merchants, 6.3M transactions, six real-world fraud typologies with ground-truth labels
- Async ingestion: semaphore-gated concurrency, exponential-backoff retry, ~330 MB → bronze
- All version-controlled with conventional commits, zero secrets in the repo

Next: Phase 2 builds the PySpark medallion transforms (silver + gold layers), turning these raw Parquet files into a queryable fraud feature store.